Vericoding

Verification-Driven Development

From behavioural features to machine-checked proofs

Gavin Mendel-Gleason · Scidonia · July 2026

Vibecoding

The status quo of LLM-driven development.

You prompt. The model emits. It runs.

What you leave behind:

- **No specification** of intended behaviour
- **No evidence** of correctness
- **No conformance regime** for the next generation

The only durable artifact is the code itself.

Vericoding

The methodology specsaver operationalises.

You write a specification. The spec survives.

What you leave behind:

- **Features** — Gherkin scenarios domain experts can review
- **Contracts** — mathematical pre/post-conditions
- **Proof obligations** — machine-checked in Rocq/Coq

Generated code is a replaceable implementation detail.

The Pipeline

Feature Gherkin Rules + Examples tables	→	**Contract** requires / ensures frames / exits / ghost	→	
Testing scenario runner per Examples row	→	**Lockdown** Coq obligations FunSpecS proofs		

Every Examples row is exercised against the contract. The same contract lowers to proof obligations.

Two Development Flows

Contract-First

> Feature → Contract → Implementation → Testing → Formal Proof

Write features and contracts first. Build the implementation against the specification. The contract is the acceptance criteria.

Retrospective

> Feature → Implementation → Testing → Contract → Testing → Formal Proof

Start with existing features and code. Write contracts that capture observed behaviour. Re-run tests under contract checking to validate. Proceed to proof.

Both flows converge on the same long-lived artifact: the contract.

Contract Language

The pure terminating fragment of Python

Every clause is a boolean-valued lambda:

- `requires(state, args)` — admissibility
- `ensures(state, args, result, new_state)` — success post
- `when(state, args)` — exception conditions
- `invariant(state)` — global legality

Static purity check rejects side effects, loops, and non-boolean returns before execution.

Anatomised as mathematical record

$$C = \langle \Sigma,\ args,\ \textbf{pre},\ \textbf{post},\\ \mathcal{X},\ \mathcal{I},\ \mathcal{D},\ \mathcal{W},\ \mathcal{R},\ \mathcal{G} \rangle$$

$\mathcal{X}$ — exceptional exits (when + ensures + frame)

$\mathcal{I}$ — invariants on every state

$\mathcal{D}$ — derived-state definitions

$\mathcal{W}$ / $\mathcal{R}$ — semantic frame

$\mathcal{G}$ — ghost state

Semantic Frames

Declare **what you write**, not what you don't.

Write paths

```
writes = {
    "state.products[sku].reserved",
    "state.invitations[token]",
    "state.reservation_log",
}
```

Derived obligations

- ✓ Every other keyed attribute unchanged
- ✓ Every other keyed row untouched
- ✓ All other event channels silent
- ✓ Deletions always rejected
- ✓ Keyed-row inserts allowed for args-resolved keys

The checker derives frame soundness from write paths alone. No "everything else unchanged" clauses needed.

Exception Exits

Exceptions are **data**, not control flow.

```
exceptions = [
    ExcExit(
        raises = InsufficientStockError,
        when   = [lambda s, a:
                    s.products[a.sku].on_hand
                  - s.products[a.sku].reserved < a.quantity],
        writes = {"state.failure_log"},
        ensures = [lambda s, a, e, s':
                     extends_by_one(
                        s.failure_log, s'.failure_log,
                        λf. f.sku == a.sku
                            ∧ f.reason == e.code)]
    )
]
```

The runner checks: correct exception raised, failure telemetry contains exactly one new event with exact fields, nothing outside the exit's writes changed.

Symmetric Projection

snap(ctx) → SpecState — used twice:

Pre-state
snap(context) before execution

Post-state
snap(context) after execution

Same function, same schema, same interpretation.
No separate "read" and "check" path.

Per Examples row:

1. materialize
   witness → temp SQLite DB
2. pre-check
   invariants + admissibility
3. execute
   service + effect-emitting wrapper
4. frame check
   outside writes unchanged
5. derived check
   derived ≡ recomputed
6. post-check
   ensures + invariants

Domain Declaration

One object → runners, CLI, conformance suite.

```
inventory = SqlDomain(
    name         = "inventory",
    package      = "examples.inventory",
    materializer = SqlMaterializer(ddl, TableSpec[]),
    projection   = SqlProjection(types, hooks),
    operations   = [
        SqlOperation(contract, wrapper,
                     witness_builder, feature_file),
        ...
    ]
)
```

scenario runners	CLI dispatch	conformance tests

Adding a domain: types + service + contracts + features + witness builders + one declaration. Everything else (materializer, projection, runner wiring, test dispatch) is derived.

Theory Stack

Services run **unmodified** on differentially-tested stubs.

Service ordinary SQLAlchemy / logging / OTel API code

Shim make_engine, make_log_capture, make_otel_capture

StubHandler operational semantics over the state model

State model TableStore / LogStore / OtelStore

Trace final interpretation (events)

Theory conformance: Stub and real library agree observationally on a differential suite; translator rejects out-of-fragment input loudly; state model and trace are pure data.

Lowering Pipeline

Contract

Python lambdas

→

introspect

shape extraction

deltas, scalars, arms

→

emit

gen_pre / gen_post

+ obligation set

→

score

coqc compile

per-obligation bisection

PROVED

machine-checked against kernel

UNKNOWN

queued for LLM proof oracle

DISPROVED

runtime counterexample found

Example: Inventory Reserve

Pre-condition:

$$\mathsf{pre}(s,a) \;=\; a.quantity > 0 \;\wedge\; a.sku \in s.products$$

Post-condition (excerpt):

$$\begin{aligned}\mathsf{post}(s,a,r,s') \;&= s'.products[a.sku].reserved \;=\; s[\,\cdot\,].reserved + a.quantity \\ &\wedge \mathsf{extends\_by\_one}(s.reservation\_log,\; s'.reservation\_log,\; \lambda e.\; e.sku = a.sku) \\ &\wedge \mathsf{implies}(crossed(s,a,s'),\; \mathsf{extends\_by\_one}(s.alert\_log,\; s'.alert\_log,\; \ldots))\end{aligned}$$

Exception exit:

$$\mathcal{X} = \{\, \langle \mathsf{InsufficientStock},\; s.available(a.sku) < a.quantity,\; \{failure\_log\},\; \mathsf{failure\_ensures} \rangle \,\}$$

Example: Invitations

Invite (insert frame):

$$\textbf{post}_{invite}(s, a, r, s') = s'.invitations[a.token].status = \text{pending} \\ \wedge\ s'.invitations[a.token].expires\_at = a.now + 7 \cdot 86400$$

Accept (multi-table delta):

$$\textbf{post}_{accept}(s, a, r, s') = s'.invitations[a.token].status = \text{accepted} \\ \wedge\ s'.members[a.user\_id].org\_id = s.invitatio \\ \wedge\ s'.users[a.user\_id].default\_org = s.invitatio$$

Design decisions for verifiability: Token is a caller-supplied argument (args-resolved insert key). now is an integer epoch argument (expiry is a pure comparison).

Verified State

Domain	Tables	Ops	Rows	Obligations
inventory	1	3	22	6/6 · 6/6 · 4/4
bank_transfer	2	1	11	7/7
invitations	3	2	9	runtime-only
Total		**6**	**42**	**23 in 23**

All four store-obligation contracts fully proved in Rocq. Trace obligations (extends_by_one) active in development.

```
Definition gen_post (sigma : sn_state)
    (vs : list sn_val) (r : Result)
    (ups : cell_updates) : Prop :=
  exists sku order quantity store_d
         on_hand_sku reserved_sku
         reorder_point_sku,
    vs = [LitString sku;
          LitString order;
          LitInt quantity] ∧
    sigma !! store_loc =
      Some (LitDict store_d) ∧
    dict_lookup_str sku store_d =
      Some (row_of on_hand_sku
                   reserved_sku
                   reorder_point_sku) ∧
    r = RVal (LitInt reserved_sku) ∧
    ups = [(store_loc,
            LitDict
              (dict_insert_str sku
                (row_of on_hand_sku
                  (reserved_sku + quantity)
                  reorder_point_sku)
                store_d))].
```

The Vericoding Promise

Vibecoding leaves

code and hope

↓

Vericoding leaves

a specification and evidence

specsaver · github.com/scidonia/specsaver